

Programming Fundamentals in C

Chapter 8: Characters and Strings

Umer Shahid

Department of Electrical Engineering, UET Lahore

November 30, 2025

Today's Agenda

1. Introduction to Characters and Strings
2. Fundamentals of Strings and Characters
3. Character-Handling Library
4. String-Conversion Functions
5. Standard Input/Output Library Functions
6. String-Manipulation Functions
7. String Comparison Functions
8. String Search Functions
9. Memory Functions
10. Other String Functions

What Will We Learn Today?

- **Characters:** The building blocks of text
- **Strings:** Collections of characters
- **Library Functions:** Tools to work with text
- **Real-world Applications:**
 - Text editors (like Notepad)
 - Word processors (like MS Word)
 - Search engines
 - Chat applications

Why is this important? Almost every program deals with text!

Understanding Programs as Characters

Everything is Characters!

Your C program itself is made of characters. When you write:

```
printf("Hello");
```

The computer reads each letter: p, r, i, n, t, f, etc.

p	r	i	n	t	f
---	---	---	---	---	---

What is a Character?

Character Constant

- A single character in **single quotes**
- Stored as an `int` value
- Examples:
 - `'A'` - uppercase letter
 - `'z'` - lowercase letter
 - `'5'` - digit character
 - `'\n'` - newline

ASCII Values

Each character has a number:

- `'A'` = 65
- `'B'` = 66
- `'a'` = 97
- `'0'` = 48
- `' '` = 32 (space)

What is a String?

Definition

A string is a **series of characters** treated as a single unit.

String Literals (in double quotes)

- "John Q. Doe" - a name
- "99999 Main Street" - an address
- "(201) 555-1212" - a phone number
- "Hello, World!" - a message

Important Difference

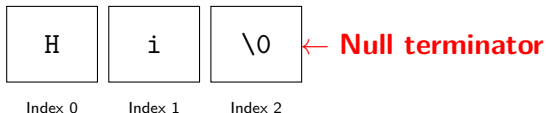
- 'A' - single character (single quotes)
- "A" - string with one character (double quotes)

The Null Character: String Terminator

Critical Concept

Every string **MUST** end with '`\0`' (null character)

String "Hi" in memory:



Warning!

Without '`\0`', printing may:

- Continue past your string (printing garbage)
- Crash your program ("segmentation fault")

Initializing Strings: Two Methods

Method 1: Character Array

```
char color[] = "blue";
```

Creates:

- 5-element array
- **Modifiable**
- Contains: 'b', 'l', 'u', 'e', '\0'

Method 2: Pointer

```
const char *colorPtr = "blue";
```

Creates:

- Pointer to string
- **NOT modifiable**
- Points to 'b' in "blue"

Alternative Array Initialization

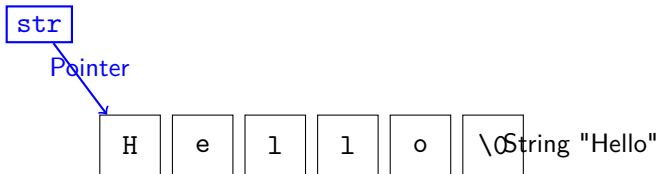
```
char color[] = {'b', 'l', 'u', 'e', '\0'};
```

Same result as `char color[] = "blue";`

Strings and Pointers

Key Concept

A string is accessed via a **pointer** to its first character.



In Code

```
char str[] = "Hello";  
// str is a pointer to 'H' (first character)
```

Reading Strings with scanf

Syntax

```
char word[20];  
scanf("%19s", word); // Read up to 19 characters
```

CRITICAL: Always Use Field Width!

- %19s reads maximum 19 characters
- Saves last position for '\0'
- **Without field width:** User can overflow array!

Why No & Needed?

```
scanf("%19s", word); // NO & before word!  
// word is already a pointer (array name)
```

Self-Check 1: Strings Basics

Question 1 (Fill-in)

A string is accessed via a _____ to the string's first character.

Self-Check 1: Strings Basics

Question 1 (Fill-in)

A string is accessed via a _____ to the string's first character.

Answer

pointer

Self-Check 1: Strings Basics

Question 1 (Fill-in)

A string is accessed via a _____ to the string's first character.

Answer

pointer

Question 2 (True/False)

The following initializes color to "blue":

```
char color[] = {'b', 'l', 'u', 'e'};
```

Self-Check 1: Strings Basics

Question 1 (Fill-in)

A string is accessed via a _____ to the string's first character.

Answer

pointer

Question 2 (True/False)

The following initializes color to "blue":

```
char color[] = {'b', 'l', 'u', 'e'};
```

Answer

FALSE! Missing '\0' - should be:

```
char color[] = {'b', 'l', 'u', 'e', '\0'};
```

Character-Handling Library: ctype.h

What Does It Do?

Provides functions to:

- **Test** characters (is it a digit? letter?)
- **Convert** characters (uppercase $\leftrightarrow$ lowercase)

How to Use

```
#include <ctype.h> // Add this at the top

int main(void) {
    char ch = 'A';
    if (isalpha(ch)) {
        printf("It's a letter!\n");
    }
}
```

Character Testing Functions (Part 1)

Function	Returns TRUE if...
<code>isdigit(c)</code>	c is a digit (0-9)
<code>isalpha(c)</code>	c is a letter (A-Z, a-z)
<code>isalnum(c)</code>	c is a letter OR digit
<code>isxdigit(c)</code>	c is hexadecimal digit (0-9, A-F, a-f)
<code>islower(c)</code>	c is lowercase letter (a-z)
<code>isupper(c)</code>	c is uppercase letter (A-Z)
<code>isspace(c)</code>	c is whitespace (space, tab, newline)

Return Values

- Returns **non-zero** (true) if condition met
- Returns **0** (false) otherwise

Character Testing Functions (Part 2)

Function	Returns TRUE if...
iscntrl(c)	c is control character (\t, \n, \r)
ispunct(c)	c is punctuation (\$, #, !, ?, etc.)
isprint(c)	c is printable (including space)
isgraph(c)	c is printable (excluding space)
isblank(c)	c is blank (space or tab)

Quick Example

```
if (ispunct('!')) { // TRUE - returns non-zero
    printf("Punctuation found!\n");
}
```

Example: isdigit, isalpha, isalnum

```
#include <stdio.h>
#include <ctype.h>

int main(void) {
    // Testing isdigit
    printf("%s\n", isdigit('8') ? "8 is a digit" : "8 is not a digit");
    printf("%s\n", isdigit('#') ? "# is a digit" : "# is not a digit");

    // Testing isalpha
    printf("%s\n", isalpha('A') ? "A is a letter" : "A is not a letter");
    printf("%s\n", isalpha('4') ? "4 is a letter" : "4 is not a letter");

    // Testing isalnum
    printf("%s\n", isalnum('A') ? "A is digit/letter" : "A is not");
    printf("%s\n", isalnum('#') ? "# is digit/letter" : "# is not");
}
```

Output

```
8 is a digit # is not a digit A is a letter 4 is not a letter A is
digit/letter # is not
```

Character Conversion Functions

Function	Description
<code>tolower(c)</code>	Converts <code>c</code> to lowercase if uppercase; otherwise returns <code>c</code> unchanged
<code>toupper(c)</code>	Converts <code>c</code> to uppercase if lowercase; otherwise returns <code>c</code> unchanged

Examples

```
char ch1 = toupper('a'); // ch1 = 'A'
char ch2 = toupper('5'); // ch2 = '5' (unchanged)
char ch3 = tolower('B'); // ch3 = 'b'
char ch4 = tolower('!'); // ch4 = '!' (unchanged)
```

Example: Case Conversion

```
#include <stdio.h>
#include <ctype.h>

int main(void) {
    printf("u converted to uppercase: %c\n", toupper('u'));
    printf("7 converted to uppercase: %c\n", toupper('7'));
    printf("L converted to lowercase: %c\n", tolower('L'));
    printf("$ converted to lowercase: %c\n", tolower('$'));
}
```

Output

```
u converted to uppercase:  U
7 converted to uppercase:  7
L converted to lowercase:  l
$ converted to lowercase:  $
```

Practical Application: Password Validation

```
#include <stdio.h>
#include <ctype.h>

int main(void) {
    char password[20];
    int hasDigit = 0, hasUpper = 0, hasLower = 0;

    printf("Enter password: ");
    scanf("%19s", password);

    for (int i = 0; password[i] != '\0'; i++) {
        if (isdigit(password[i])) hasDigit = 1;
        if (isupper(password[i])) hasUpper = 1;
        if (islower(password[i])) hasLower = 1;
    }

    if (hasDigit && hasUpper && hasLower) {
        printf("Strong password!\n");
    } else {
        printf("Weak: Need digit, uppercase & lowercase\n");
    }
}
```

Self-Check 2: Character Functions

Question 1 (Multiple Choice)

Which function is: "Returns true if character is digit OR letter"?

a) `isdigit` b) `isalpha` c) `isalnum` d) `isxdigit`

Self-Check 2: Character Functions

Question 1 (Multiple Choice)

Which function is: "Returns true if character is digit OR letter"?

a) `isdigit` b) `isalpha` c) `isalnum` d) `isxdigit`

Answer

c) `isalnum`

Self-Check 2: Character Functions

Question 1 (Multiple Choice)

Which function is: "Returns true if character is digit OR letter"?

a) isdigit b) isalpha c) isalnum d) isxdigit

Answer

c) isalnum

Question 2

What does this print?

```
printf("%s\n", isalpha('m') ? "m is a letter" : "m is not");
```

Self-Check 2: Character Functions

Question 1 (Multiple Choice)

Which function is: "Returns true if character is digit OR letter"?

a) isdigit b) isalpha c) isalnum d) isxdigit

Answer

c) isalnum

Question 2

What does this print?

```
printf("%s\n", isalpha('m') ? "m is a letter" : "m is not");
```

Answer

m is a letter

String-Conversion Functions: `stdlib.h`

Purpose

Convert strings of digits to numeric values

Function	Converts to...
<code>strtod()</code>	double
<code>strtol()</code>	long int
<code>strtoul()</code>	unsigned long int

Why Needed?

```
char numStr[] = "123";  
// numStr is a STRING, not a number!  
// Can't do math: numStr + 5 won't work  
// Need to convert to int first
```

Function strtod: String to Double

Syntax

```
double strtod(const char *str, char **endPtr);
```

- `str`: String to convert
- `endPtr`: Points to first character after number
- Returns: Converted double value

Example

```
const char *string = "51.2% are admitted";  
char *stringPtr = NULL;  
  
double d = strtod(string, &stringPtr);  
// d = 51.2  
// stringPtr points to "% are admitted"
```

Function strtol: String to Long Integer

Syntax

```
long strtol(const char *str, char **endPtr, int base);
```

- base: Number system (0, 2-36)
 - 0 = auto-detect (octal, decimal, hex)
 - 10 = decimal
 - 16 = hexadecimal

Example

```
const char *string = "-1234567abc";  
char *remainderPtr = NULL;  
  
long x = strtol(string, &remainderPtr, 0);  
// x = -1234567  
// remainderPtr points to "abc"
```

Function strtoul: String to Unsigned Long

Similar to strtol

```
unsigned long strtoul(const char *str, char **endPtr, int base);
```

Converts to unsigned long int (positive numbers only)

Example

```
const char *string = "1234567abc";  
char *remainderPtr = NULL;  
  
unsigned long x = strtoul(string, &remainderPtr, 0);  
// x = 1234567  
// remainderPtr points to "abc"
```

Self-Check 3: String Conversion

Question 1

Why would a function parameter be `char **`?

Self-Check 3: String Conversion

Question 1

Why would a function parameter be `char **`?

Answer

To modify a `char *` pointer in the caller. It's a pointer to a pointer, allowing the function to change where the caller's pointer points.

Self-Check 3: String Conversion

Question 1

Why would a function parameter be `char **`?

Answer

To modify a `char *` pointer in the caller. It's a pointer to a pointer, allowing the function to change where the caller's pointer points.

Question 2 (Multiple Choice)

Which statement about `strtol` is FALSE?

- a) Returns 0 if unable to convert
- b) Takes 3 arguments: string, pointer, base
- c) Ignores leading whitespace
- d) Uses `char**` to return numeric value

Self-Check 3: String Conversion

Question 1

Why would a function parameter be `char **`?

Answer

To modify a `char *` pointer in the caller. It's a pointer to a pointer, allowing the function to change where the caller's pointer points.

Question 2 (Multiple Choice)

Which statement about `strtol` is FALSE?

- a) Returns 0 if unable to convert
- b) Takes 3 arguments: string, pointer, base
- c) Ignores leading whitespace
- d) Uses `char**` to return numeric value

Answer

d) FALSE - Uses `char**` to return location after conversion, not the number

String I/O Functions: `stdio.h`

Function	Description
<code>getchar()</code>	Read one character from keyboard
<code>putchar(c)</code>	Print one character
<code>fgets(s,n,stdin)</code>	Read string (up to n-1 chars)
<code>puts(s)</code>	Print string + newline
<code>sprintf(s,...)</code>	Print formatted to string
<code>sscanf(s,...)</code>	Read formatted from string

Function fgets: Read Line Safely

Syntax

```
char *fgets(char *str, int n, FILE *stream);
```

- Reads up to $n-1$ characters
- Stops at newline or EOF
- Includes newline in string
- Adds `'\0'` automatically
- Use `stdin` for keyboard input

Example

```
char sentence[80];  
printf("Enter text: ");  
fgets(sentence, 80, stdin);  
// Reads up to 79 characters safely
```

Functions getchar and putchar

getchar()

```
int getchar(void);
```

- Reads ONE character
- Returns as int
- Returns EOF at end

putchar(c)

```
int putchar(int c);
```

- Prints ONE character
- Returns printed char

Reading String Character-by-Character

```
char sentence[80];  
int c, i = 0;  
while ((i < 79) && (c = getchar()) != '\n') {  
    sentence[i++] = c;  
}  
sentence[i] = '\0'; // Don't forget terminator!
```

Function sprintf: Format to String

Like printf, but stores in string

```
int sprintf(char *str, const char *format, ...);
```

Example

```
char s[80];  
int age = 20;  
double gpa = 3.75;  
  
sprintf(s, "Age: %d, GPA: %.2f", age, gpa);  
// s now contains: "Age: 20, GPA: 3.75"  
  
printf("Result: %s\n", s);
```

Use Case

Building strings for display, logging, or file output

Function sscanf: Read from String

Like scanf, but reads from string

```
int sscanf(const char *str, const char *format, ...);
```

Example

```
char data[] = "25 87.5";  
int age;  
double score;  
  
sscanf(data, "%d %lf", &age, &score);  
// age = 25  
// score = 87.5  
  
printf("Age: %d, Score: %.1f\n", age, score);
```

Use Case

Parsing data from files or formatted strings

Self-Check 4: I/O Functions

Question 1 (Multiple Choice)

Which function "Prints character and returns it as integer"?

- a) getchar b) sprintf c) puts d) putchar

Self-Check 4: I/O Functions

Question 1 (Multiple Choice)

Which function "Prints character and returns it as integer"?

- a) getchar b) sprintf c) puts d) putchar

Answer

d) putchar

Self-Check 4: I/O Functions

Question 1 (Multiple Choice)

Which function "Prints character and returns it as integer"?

- a) getchar b) sprintf c) puts d) putchar

Answer

d) putchar

Question 2 (True/False)

getchar() returns a char

Self-Check 4: I/O Functions

Question 1 (Multiple Choice)

Which function "Prints character and returns it as integer"?

- a) getchar b) sprintf c) puts d) putchar

Answer

d) putchar

Question 2 (True/False)

getchar() returns a char

Answer

FALSE! Returns int to support EOF indicator (-1)

String-Handling Library: string.h

Functions for:

- **Copying** strings
- **Concatenating** (joining) strings
- **Comparing** strings
- **Searching** strings
- **Tokenizing** (splitting) strings
- Getting string **length**

Category	Functions
Copy	strcpy, strncpy
Concatenate	strcat, strncat
Compare	strcmp, strncmp
Search	strchr, strstr, strtok
Length	strlen

Function strcpy: Copy String

Syntax

```
char *strcpy(char *dest, const char *src);
```

Copies src string into dest array

Example

```
char x[] = "Happy Birthday";  
char y[25]; // Must be large enough!  
  
strcpy(y, x);  
printf("%s\n", y); // Prints: Happy Birthday
```

Function strcpy: Copy String

Syntax

```
char *strcpy(char *dest, const char *src);
```

Copies src string into dest array

CRITICAL WARNING!

dest array **MUST** be large enough for:

- Source string
- Null terminator '\0'

Otherwise: Buffer overflow → Program crash!

Function strncpy: Copy n Characters

Syntax

```
char *strncpy(char *dest, const char *src, size_t n);
```

Copies at most n characters from src

Example

```
char x[] = "Happy Birthday to You";  
char z[15];  
  
strncpy(z, x, 14); // Copy first 14 chars  
z[14] = '\0'; // MUST add terminator!  
  
printf("%s\n", z); // Prints: Happy Birthday
```

Function strncpy: Copy n Characters

Syntax

```
char *strncpy(char *dest, const char *src, size_t n);
```

Copies at most `n` characters from `src`

Important!

`strncpy` may NOT add `'\0'` if $n \leq \text{string length}$!

Always manually add `'\0'` to be safe!

Visual: strcpy vs strncpy

Original string: "Hello"

strcpy:

H	e	l	l	o	\0
---	---	---	---	---	----

All copied + \0

strncpy(n=3):

H	e	l	?
---	---	---	---

NO \0 added!

Solution with strncpy

```
strncpy(dest, src, n);  
dest[n] = '\0'; // Always add this!
```

Function strcat: Concatenate Strings

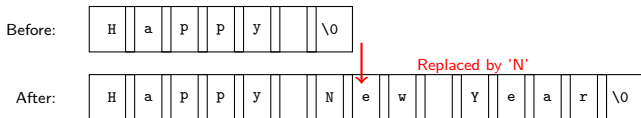
Syntax

```
char *strcat(char *dest, const char *src);
```

Appends src to end of dest

Example

```
char s1[20] = "Happy ";  
char s2[] = "New Year";  
  
strcat(s1, s2);  
printf("%s\n", s1); // Prints: Happy New Year
```



Function strncat: Concatenate n Characters

Syntax

```
char *strncat(char *dest, const char *src, size_t n);
```

Appends at most `n` characters + adds `'\0'` automatically

Example

```
char s1[40] = "Happy New Year ";  
char s3[40] = "";  
  
strncat(s3, s1, 6); // Append first 6 chars  
printf("%s\n", s3); // Prints: Happy  
  
strcat(s3, s1); // Append all of s1  
printf("%s\n", s3); // Prints: Happy Happy New Year
```

Advantage over strcat

`strncat` automatically adds `'\0'` - safer than `strncpy`!

Self-Check 5: String Manipulation

Question 1 (Multiple Choice)

Which statement about `strcat` is FALSE?

- a) Appends second string to first
- b) First char of `src` placed after `'\0'` of `dest`
- c) Must ensure `dest` array is large enough
- d) Automatically adds `'\0'`

Self-Check 5: String Manipulation

Question 1 (Multiple Choice)

Which statement about `strcat` is FALSE?

- a) Appends second string to first
- b) First char of `src` placed after `'\0'` of `dest`
- c) Must ensure `dest` array is large enough
- d) Automatically adds `'\0'`

Answer

- b) FALSE - First char **replaces** the `'\0'` of `dest`

Self-Check 5: String Manipulation

Question 1 (Multiple Choice)

Which statement about `strcat` is FALSE?

- a) Appends second string to first
- b) First char of `src` placed after `'\0'` of `dest`
- c) Must ensure `dest` array is large enough
- d) Automatically adds `'\0'`

Answer

- b) FALSE - First char **replaces** the `'\0'` of `dest`

Question 2 (Fill-in)

`strcpy` requires destination array to be _____

Self-Check 5: String Manipulation

Question 1 (Multiple Choice)

Which statement about `strcat` is FALSE?

- a) Appends second string to first
- b) First char of `src` placed after `'\0'` of `dest`
- c) Must ensure `dest` array is large enough
- d) Automatically adds `'\0'`

Answer

- b) FALSE - First char **replaces** the `'\0'` of `dest`

Question 2 (Fill-in)

`strcpy` requires destination array to be _____

Answer

large enough to store string + terminating null character

Function strcmp: Compare Strings

Syntax

```
int strcmp(const char *s1, const char *s2);
```

Return Values

- **0**: Strings are equal
- **Negative**: s1 < s2
- **Positive**: s1 > s2

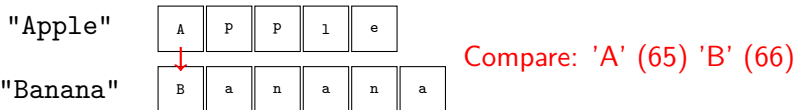
Example

```
const char *s1 = "Hello";  
const char *s2 = "Hello";  
const char *s3 = "World";  
  
printf("%d\n", strcmp(s1, s2)); // 0 (equal)  
printf("%d\n", strcmp(s1, s3)); // Negative (H < W)  
printf("%d\n", strcmp(s3, s1)); // Positive (W > H)
```

How String Comparison Works

Lexicographical Comparison

Compares character by character using ASCII/Unicode values



Result: "Apple" "Banana"

Important Facts

- Uppercase letters lowercase letters
- 'A' = 65, 'Z' = 90, 'a' = 97, 'z' = 122
- "Apple" "apple" (because 'A' 'a')

Function strncmp: Compare n Characters

Syntax

```
int strncmp(const char *s1, const char *s2, size_t n);
```

Compares up to n characters

Example

```
const char *s1 = "Happy New Year";  
const char *s3 = "Happy Holidays";  
  
printf("%d\n", strcmp(s1, s3)); // Positive (N > H)  
printf("%d\n", strncmp(s1, s3, 6)); // 0 (first 6 equal)  
printf("%d\n", strncmp(s1, s3, 7)); // Positive (N > H)
```

"Happy New Year"



First 6 characters match

"Happy Holidays"

Practical: Sorting Names

```
#include <stdio.h>
#include <string.h>

int main(void) {
    char names[3][20] = {"Zara", "Ali", "Bilal"};
    char temp[20];

    // Bubble sort
    for (int i = 0; i < 2; i++) {
        for (int j = 0; j < 2-i; j++) {
            if (strcmp(names[j], names[j+1]) > 0) {
                strcpy(temp, names[j]);
                strcpy(names[j], names[j+1]);
                strcpy(names[j+1], temp);
            }
        }
    }

    printf("Sorted names:\n");
    for (int i = 0; i < 3; i++) {
        printf("%s\n", names[i]);
    }
}
```

Output

Self-Check 6: String Comparison

Question 1 (Multiple Choice)

Which is FALSE about `strcmp`?

- a) Returns 0 if strings equal
- b) Returns negative if first second
- c) Compares character-by-character
- d) Compares characters after `'\0'`

Self-Check 6: String Comparison

Question 1 (Multiple Choice)

Which is FALSE about `strcmp`?

- a) Returns 0 if strings equal
- b) Returns negative if first second
- c) Compares character-by-character
- d) Compares characters after `'\0'`

Answer

- d) FALSE - Does NOT compare after `'\0'`

Self-Check 6: String Comparison

Question 1 (Multiple Choice)

Which is FALSE about strcmp?

- a) Returns 0 if strings equal
- b) Returns negative if first second
- c) Compares character-by-character
- d) Compares characters after '\0'

Answer

- d) FALSE - Does NOT compare after '\0'

Question 2

How do comparison functions know 'A' comes before 'B'?

Self-Check 6: String Comparison

Question 1 (Multiple Choice)

Which is FALSE about strcmp?

- a) Returns 0 if strings equal
- b) Returns negative if first second
- c) Compares character-by-character
- d) Compares characters after '\0'

Answer

- d) FALSE - Does NOT compare after '\0'

Question 2

How do comparison functions know 'A' comes before 'B'?

Answer

By comparing numeric ASCII/Unicode values (A=65, B=66). This is called lexicographical comparison.

String Search Functions Overview

Function	Searches For...
<code>strchr</code>	First occurrence of character
<code>strrchr</code>	Last occurrence of character
<code>strstr</code>	First occurrence of substring
<code>strpbrk</code>	First char from set
<code>strcspn</code>	Length until char from set
<code>strspn</code>	Length of chars from set
<code>strtok</code>	Split into tokens

Function strchr: Find Character

Syntax

```
char *strchr(const char *str, int ch);
```

Returns pointer to first occurrence of `ch`, or `NULL` if not found

Example

```
const char *string = "This is a test";

if (strchr(string, 'a') != NULL) {
    printf("'a' found!\n");
}

if (strchr(string, 'z') == NULL) {
    printf("'z' not found!\n");
}
```

Output

```
'a' found!
```

```
'z' not found!
```

Function strrchr: Find Last Character

Syntax

```
char *strrchr(const char *str, int ch);
```

Returns pointer to **last** occurrence of `ch`

Example

```
const char *s = "A zoo has many animals including zebras";  
  
char *result = strrchr(s, 'z');  
printf("Last 'z': %s\n", result);
```

Output

Last 'z': zebras

"A zoo has many animals including zebras"

↓
First 'z'

↓
Last 'z' (returned)

Function strstr: Find Substring

Syntax

```
char *strstr(const char *haystack, const char *needle);
```

Finds first occurrence of needle string in haystack

Example

```
const char *string1 = "abcdefabcdef";  
const char *string2 = "def";  
  
char *result = strstr(string1, string2);  
printf("Found at: %s\n", result);
```

Output

Found at: defabcdef

"abcdef**def**abcdef"



First "def" found here

Function `strcspn`: Span Excluding Characters

Syntax

```
size_t strcspn(const char *str, const char *reject);
```

Returns length of initial segment NOT containing any chars from `reject`

Example

```
const char *string1 = "The value is 3.14159";  
const char *string2 = "1234567890"; // Digits  
  
size_t len = strcspn(string1, string2);  
printf("Length before first digit: %zu\n", len);
```

Output

Length before first digit: 13

"The value is 3.14159"
↑ 13 characters before first digit

Function `strspn`: Span Including Characters

Syntax

```
size_t strspn(const char *str, const char *accept);
```

Returns length of initial segment containing ONLY chars from `accept`

Example

```
const char *string1 = "The value is 3.14159";  
const char *string2 = "aehi lsTuv"; // Valid chars  
  
size_t len = strspn(string1, string2);  
printf("Length of initial valid segment: %zu\n", len);
```

Output

Length of initial valid segment: 13

Function strtok: Tokenize String

What is a Token?

A sequence of characters separated by **delimiters**

Example: "Hello World" has 2 tokens (delimiter = space)

First Call

```
char string[] = "This is a sentence";  
char *tokenPtr = strtok(string, " "); // Use space as delimiter  
printf("%s\n", tokenPtr); // Prints: This
```

Subsequent Calls

```
tokenPtr = strtok(NULL, " "); // NULL = continue same string  
printf("%s\n", tokenPtr); // Prints: is  
  
tokenPtr = strtok(NULL, " ");  
printf("%s\n", tokenPtr); // Prints: a
```

strtok: Complete Example

```
#include <stdio.h>
#include <string.h>

int main(void) {
    char string[] = "This is a sentence with 7 tokens";

    printf("Original: %s\n\n", string);
    printf("Tokens:\n");

    // First call with string
    char *tokenPtr = strtok(string, " ");

    // Loop until no more tokens
    while (tokenPtr != NULL) {
        printf("%s\n", tokenPtr);
        tokenPtr = strtok(NULL, " "); // Subsequent calls with NULL
    }
}
```

Output

```
Original: This is a sentence with 7 tokens Tokens: This is a sentence
with 7 tokens
```

Self-Check 7: Search Functions

Question 1 (Multiple Choice)

Which function: "Locates first occurrence of string s2 in s1"?

- a) `strpbrk` b) `strstr` c) `strspn` d) `strcspn`

Self-Check 7: Search Functions

Question 1 (Multiple Choice)

Which function: "Locates first occurrence of string s2 in s1"?

- a) `strpbrk` b) `strstr` c) `strspn` d) `strcspn`

Answer

- b) `strstr`

Self-Check 7: Search Functions

Question 1 (Multiple Choice)

Which function: "Locates first occurrence of string s2 in s1"?

- a) `strpbrk` b) `strstr` c) `strspn` d) `strcspn`

Answer

- b) `strstr`

Question 2 (Fill-in)

In `strtok`, a _____ is a sequence of characters separated by delimiters.

Self-Check 7: Search Functions

Question 1 (Multiple Choice)

Which function: "Locates first occurrence of string s2 in s1"?

- a) `strpbrk` b) `strstr` c) `strspn` d) `strcspn`

Answer

b) `strstr`

Question 2 (Fill-in)

In `strtok`, a _____ is a sequence of characters separated by delimiters.

Answer

token

Memory Functions: Working with Raw Memory

Key Difference from String Functions

- String functions stop at '`\0`'
- Memory functions work with **blocks of bytes**
- Don't care about null terminators
- Can manipulate ANY data type

Function	Purpose
<code>memcpy</code>	Copy memory block
<code>memmove</code>	Copy (safe for overlap)
<code>memcmp</code>	Compare memory blocks
<code>memchr</code>	Find byte in memory
<code>memset</code>	Set bytes to value

Function memcpy: Copy Memory

Syntax

```
void *memcpy(void *dest, const void *src, size_t n);
```

Copies `n` bytes from `src` to `dest`

Example

```
char s1[17];  
char s2[] = "Copy this string";  
  
memcpy(s1, s2, 17); // Copy 17 bytes (includes \0)  
printf("%s\n", s1); // Prints: Copy this string
```

Warning

- Undefined behavior if `src` and `dest` overlap!
- Use `memmove` for overlapping regions
- Faster than `strcpy` when size is known

Function memmove: Safe Overlap Copy

Syntax

```
void *memmove(void *dest, const void *src, size_t n);
```

Like memcpy, but **safe for overlapping** memory

Example: Copy Within Same Array

```
char x[] = "Home Sweet Home";  
  
printf("Before: %s\n", x);  
memmove(x, &x[5], 10); // Copy "Sweet Home" to start  
printf("After: %s\n", x);
```

Output

Before: Home Sweet Home

After: Sweet Home Home

Copy this

Before: H o m e S w e e t H o m e

Function memcmp: Compare Memory

Syntax

```
int memcmp(const void *s1, const void *s2, size_t n);
```

Compares first n bytes of s1 and s2

Example

```
char s1[] = "ABCDEFGFG";  
char s2[] = "ABCDXYZ";  
  
printf("%d\n", memcmp(s1, s2, 4)); // 0 (first 4 equal)  
printf("%d\n", memcmp(s1, s2, 7)); // -1 (E < X)  
printf("%d\n", memcmp(s2, s1, 7)); // 1 (X > E)
```

"ABCDEFGFG"
| | | |
"ABCDXYZ"

First 4 match

Position 5: E $\neq$ X

Function memchr: Find Byte

Syntax

```
void *memchr(const void *s, int c, size_t n);
```

Searches for byte `c` in first `n` bytes of `s`

Example

```
const char *s = "This is a string";  
  
char *result = (char *)memchr(s, 'r', 16);  
printf("Found: %s\n", result);
```

Output

Found: ring

Function memchr: Find Byte

Syntax

```
void *memchr(const void *s, int c, size_t n);
```

Searches for byte `c` in first `n` bytes of `s`

Difference from strchr

- `strchr`: Stops at `'\0'`
- `memchr`: Searches exactly `n` bytes (includes data after `'\0'`)

Function memset: Initialize Memory

Syntax

```
void *memset(void *s, int c, size_t n);
```

Sets first `n` bytes of `s` to value `c`

Example: Initialize Array to Zeros

```
int numbers[5] = {1, 2, 3, 4, 5};  
// Before  
for (int i = 0; i < 5; i++) {  
    printf("%d ", numbers[i]); // 1 2 3 4 5  
}  
memset(numbers, 0, sizeof(numbers)); // Set all to 0  
// After  
for (int i = 0; i < 5; i++) {  
    printf("%d ", numbers[i]); // 0 0 0 0 0  
}
```

Practical: Using memset for Arrays

```
int main(void) {  
    char buffer[100];  
    // Clear buffer to all zeros  
    memset(buffer, 0, sizeof(buffer));  
    // Or set to specific character  
    memset(buffer, '*', 10);  
    buffer[10] = '\\0';  
    printf("Buffer: %s\\n", buffer); // *****  
}
```

Performance Tip

memset is often optimized by compiler - faster than loops!

Function strlen: Get String Length

Syntax

```
size_t strlen(const char *s);
```

Returns number of characters **before** `'\0'` (not including it)

Example

```
char str1[] = "Hello";  
char str2[] = "Programming in C";  
  
printf("Length of '%s': %zu\n", str1, strlen(str1));  
printf("Length of '%s': %zu\n", str2, strlen(str2));
```

Output

```
Length of 'Hello': 5  
Length of 'Programming in C': 16
```

Important